Gemma 4 Is Less Self-Biased Than It Looks

Sugam Panthi University of Southern Mississippi

github.com/Vein05/gemma4-paper-track Supplement: `rubber-stamp-supplement.pdf` Kaggle paper track

Abstract

Coding agents often decide whether a patch is finished by asking a model, often the one that wrote it. Studies of self-preference compare a judge's scores on its own outputs with its scores on other models' outputs, and the closest study reports that the preference shrinks but remains against writers as strong as the judge. We ask Gemma 4 31B and eight other models to score SWE-bench patches, and we grade every patch with the hidden tests. By the usual measure, Gemma trusts its own failing patches far more than other writers' failing patches. Against seven writers as strong as Gemma, about half of that gap remains, and Gemma gives their failing patches the top score as often as its own. The larger problem is the patches. Most of Gemma's failing patches are wrong fixes that pass every existing test SWE-bench runs, and so are seven in ten failing patches from 32 public models in the same agent. The share is higher for stronger models. On about one issue in ten, the correct fix breaks an existing test. When Gemma is shown such a test, it obeys the test and undoes the correct patch. When Gemma first labels such tests as outdated, most of the correct patches survive. We release every agent and judge trace on GitHub and Hugging Face.

1 Introduction

In matplotlib issue 24570, Gemma 4 31B wrote a patch that fixes the reported alignment bug but also swaps the left and right cases of a shared helper. When we later asked Gemma whether this patch would pass the project's hidden tests, it gave the highest score on our 0 to 100 scale and called the patch "a simple swap" that "correctly implements the expected behavior". GLM-4.5 scored the same patch 20 because it "breaks VPacker by reversing left and right", and the hidden tests for left and right alignment fail. Coding agents make this call at the end of every task, and many make it by asking the model that wrote the patch.

The standard test of this risk compares a judge's scores on its own outputs with its scores on other models' outputs. Pombal et al. (2026) ran this test on contest programs with executed labels. Most judges favored their own programs, and the preference was larger when the judge recognized its own output. Gemma 3 12B and 27B were among the exceptions. But errors from strong writers are harder for a verifier to detect (Zhou et al., 2025), so a judge compared with weaker writers will seem to favor itself even if it never recognizes its own work. Pombal et al. matched writers on ability and found that the preference "attenuates in places but never disappears".

We ran this test on Gemma 4 31B with SWE-bench patches, against two groups of public agents. Against five writers of mixed strength, Gemma scores its own failing patches 16.3 points further above judges from other model families than it scores the other writers' failing patches. Against the seven writers that solve about as many issues as Gemma, the gap is 8.7 points. Gemma gives these seven writers' failing patches the top score of 100 as often as its own, and the weakest writer's a quarter as often.

About half of what looks like self-preference is thus trust in any equally strong writer's failing patch. Most of Gemma's own failing patches are wrong fixes: the issue's tests fail while every other existing test passes. Other agents fail the same way. Across 32 public models in the same agent, 71% of the failing patches that change code pass every existing test that SWE-bench runs, and the share rises with the model's strength. So neither Gemma's score nor the existing tests catch most failures. The existing tests can also mislead.

On about one issue in ten, the correct fix itself breaks an existing test that asserts the old behavior. When Gemma is shown such a test, it obeys the test and undoes a correct fix.

We make the following contributions:

- **Gemma 4's self-preference halves against equally strong writers.** Its extra trust in its own failing patches falls from 16.3 points against five writers of mixed strength to 8.7 against seven of its own strength, and it gives the seven's failing patches the top score as often as its own (Section 3.1).
- **Most failing patches pass every existing test, and more of them as agents get stronger.** Across 32 public models the share is 71%. Gemma gives most of its own wrong fixes the top score, and the existing tests stop few of them (Section 3.2).
- **On one issue in ten, the correct fix breaks an existing test.** When Gemma is shown such a test, it undoes the correct patch, 12 times in 13 on Lite. Having Gemma label these tests first spares most of the correct patches. We release the 45 issues (Section 3.3).

2 Setup

Patches. The main data set is 193 held-out issues, a random sample of SWE-bench Verified issues (Jimenez et al., 2023) outside SWE-bench Verified Mini (MariusHobbhahn). We call the model that wrote a patch its writer. The 4-bit competition checkpoint of Gemma 4 31B made one attempt per issue in mini-SWE-agent v1.7.0 (SWE-agent) at temperature 0, and we graded its patches with SWE-bench's grading code. Other writers' patches are public mini-SWE-agent submissions (SWE-bench experiments) with their official labels. Five writers of mixed strength stand for the usual comparison: GLM-4.5, gpt-oss-120b, Kimi-K2, Llama-4-Maverick and Qwen3-Coder. The seven equally strong writers are every public writer whose share of issues solved is within 8 points of Gemma's: GLM-4.5 and Kimi-K2 again, o3, o4-mini, GPT-5 mini, GPT-5 nano and Gemini 2.5 Pro. We removed test files from every patch and dropped very long patches, leaving 180 from Gemma.

Two more data sets vary the run and the language. Another team ran Gemma 4 31B on the 300 SWE-bench Lite issues, in mini-SWE-agent 2.2.1 at full precision with thinking on. We ran Gemma on 120 Go, Ruby and PHP issues from SWE-bench Multilingual (SWE-bench) in mini-SWE-agent 2.0.0 with thinking on. There the other writers are four recent models, GLM-5, Kimi-K2.5, DeepSeek-V3.2 and MiniMax-M2.5.

Judges. Nine models scored every held-out patch: Gemma 4 31B, gpt-oss-20b, Qwen3.5-9B, GLM-4.7-Flash and the five mixed-strength writers. For the five writers that are only in the second group, they scored the failing patches. Each judge saw the issue and one patch, without its writer, and scored the chance that the patch passes the hidden tests from 0 to 100. Calls went through OpenRouter at temperature 0 with reasoning off (gpt-oss at low effort). Fewer judges scored the Lite and Go, Ruby and PHP patches, chosen before any score was read.

Measures. A judge's score on a failing patch says little alone, because some failing patches look right to everyone. We therefore measure how far a judge scores each patch above the mean of the outside judges, every judge from outside its own model family and the writer's. Extra trust is how much further above them a judge scores one group of failing patches than another group. We report 95% intervals from 2,000 bootstrap resamples of issues.

SWE-bench grades a patch with two groups of tests. The issue's tests fail before the reference fix and pass after it. The other existing tests must keep passing. We call a failing patch a wrong fix when the issue's tests fail and every other test passes, and a regression when it breaks one of the other tests. SWE-bench takes both groups from the test files that the reference pull request touches, so "every existing test" means every test on that list. We count wrong fixes among the failing patches that change code, because some hold no fix. The count across agents uses SWE-bench's public test reports for 32 models run in mini-SWE-agent on all 500 Verified issues.

Plans. We committed a written plan, with the rule for reading its result, before computing each planned test. The rule that picks the equally strong writers was fixed before their patches were scored. The split by failure type and the counts across public models and of outdated tests were chosen after we saw results, so they are exploratory.

3 Results

3.1 Gemma's self-preference halves against equally strong writers

Against the mixed-strength writers, Gemma's extra trust in its own failing patches is 16.3 points, and about the same on Lite (Figure 1b). Weak writers supply much of this gap. Gemma gives 100 to most of its own failing patches and to a quarter or less of the two weakest writers', and these two supply most of the group's failing patches (Figure 1a).

Against the equally strong writers, Gemma gives 100 to 59% of its own failing patches and to 57% of theirs, a difference within noise (Figure 1a; Supplement, Section C). Its extra trust in its own failing patches is 8.7 points, about half of the gap above, with a 95% interval above zero. Most of the seven are OpenAI models, so we average within each model family first.

The size of the remaining gap depends on which writers are compared. Gemma scores Gemini 2.5 Pro's failing patches as far above the outside judges as its own, and the four OpenAI writers' near or below them. Without Gemini the gap is larger, and without the OpenAI writers its interval includes zero. We also could not tell whether the remaining gap follows the content of Gemma's patches or Gemma recognizing them (Supplement, Section D).

Gemma trusts stronger writers' failing patches in Go, Ruby and PHP too. There every other writer solves more issues than Gemma, and Gemma gives their failing patches 100 nearly as often as its own.

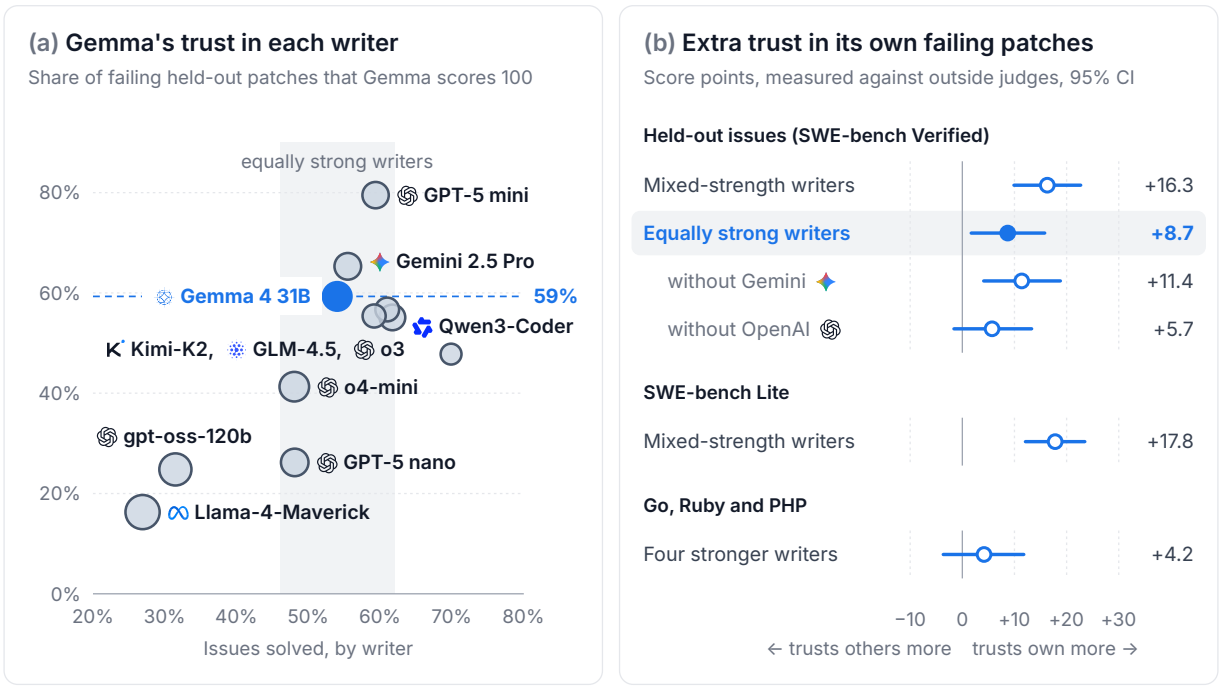


Figure 1. Gemma 4 31B trusts failing patches more when their writer is stronger. (a) Held-out issues. The shaded band holds the equally strong writers, and dot area shows the number of failing patches. (b) Gemma's extra trust in its own failing patches, with 95% intervals, halves against the equally strong writers.

3.2 Most failing patches pass every existing test, and more of them as agents get stronger

Most of Gemma's own failing patches are wrong fixes. We sorted its failing held-out patches by failure type (Supplement, Section C). A few hold no fix: they only add a reproduction script, and Gemma scores each of them 0. Of the 74 that change code, 44 are wrong fixes and the rest break an existing test. Gemma gives about three quarters of the wrong fixes the top score.

Checks that Gemma runs itself miss most of the same failures. We tried thinking before scoring, Gemma's own reproduction script, a shell in which it must write a failing test, and fine-tuning on labeled patches (Supplement, Section E). Gemma's thinking mode did not help, although Chen et al. (2025) found that long chains of thought before evaluation reduce harmful self-preference.

Other agents' failing patches are mostly wrong fixes too. Across the 32 public models, 71% of the failing patches that change code are wrong fixes (Figure 2). The share rises with the model's strength, from 64% among models that solve about half of the issues to 81% among the strongest. A check built on these tests can therefore catch at most 3 in 10 failures, and fewer for stronger models.

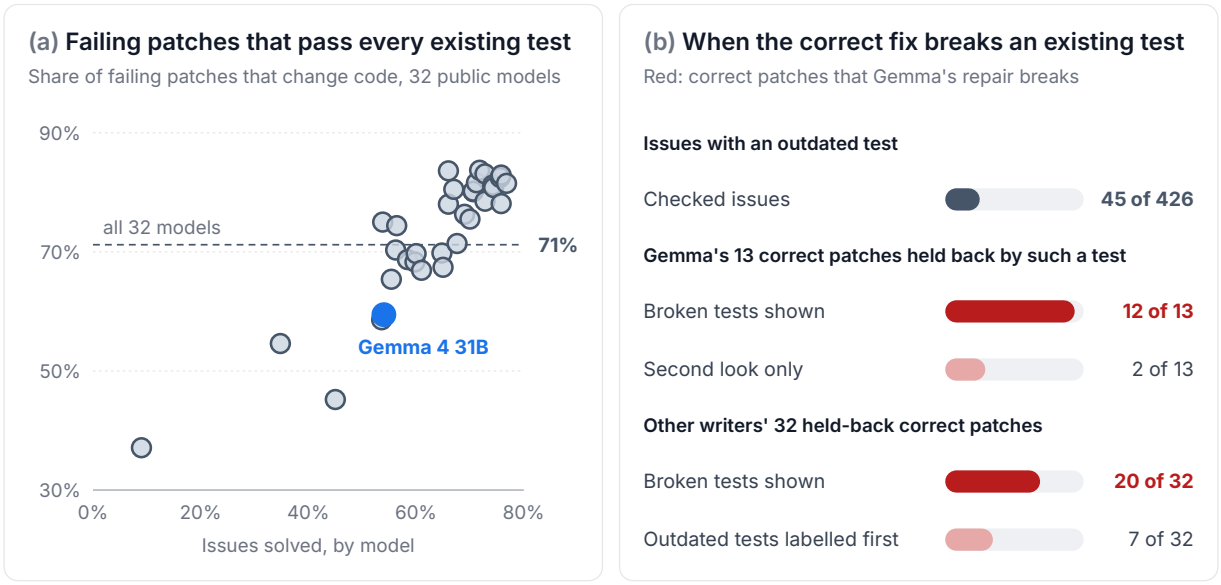


Figure 2. (a) Stronger models' failing patches pass every existing test more often. Each dot is one public model. Gemma's point uses the held-out issues. (b) How often an outdated test occurs, and how many correct patches Gemma breaks when asked to repair them (Section 3.3).

3.3 On one issue in ten, the correct fix breaks an existing test

Existing tests can still catch regressions. Our check runs the existing tests near the changed code, on SWE-bench's list or not, and holds back a patch that breaks one. A held-back patch gets one more Gemma agent run, the repair. In one condition Gemma sees the broken tests and must make them pass and keep the issue solved without editing test files. In the other, the second look, it is only told that the patch may have mistakes (Supplement, Section F).

The check holds back most of Gemma's failing held-out patches that break a listed test, and only 4 of its 44 wrong fixes. When Gemma is shown the broken tests, it repairs about half of the held-back failing patches, twice as many as with a second look (Table 1). On Gemma's Lite patches the same repair fixed almost none. Half of those patches are wrong fixes: the tests they break are outside SWE-bench's list, and making them

pass does not fix the issue.

Table 1. The repair on two sets of Gemma's patches. Repaired counts leave out one Lite issue whose test fails at random.

	Held-out, tests	Held-out, second look	Lite, tests	Lite, second look
Held-back failing patches	26	26	30	30
Repaired	14	7	1	0
Held-back correct patches	9	9	14	14
Broken by the repair	4	1	12	2
Net change in passing patches	+10	+6	−11	−2

The larger cost falls on correct patches, because some existing tests are outdated. We call a test outdated when the reference fix breaks it and the reference pull request edits or deletes it. Our check also ran on the reference fix of each issue, and an outdated test held it back on 45 of 426 issues, about one in ten (Figure 2b). This is a lower bound, because the check runs few test files per issue. On the held-out issues among them, no patch that keeps an outdated test passing is correct.

An agent told to keep existing tests passing therefore undoes correct work. On Lite, an outdated test held back 13 of the 14 correct patches in Table 1. Gemma broke all but one of them when shown the test, against 2 with a second look. On xarray issue 5131 it wrote that "the constraints of the task (making tests pass without editing them) force me to restore it".

The repair lacks a rule for which tests may change, which Agentless (Xia et al., 2024) leaves to the model. We tested one rule on the mixed-strength writers' held-back patches. Gemma first labels each broken test as outdated or a real regression, and the repair ignores the outdated ones. Correct patches broken fell from 20 to 7, but fewer failing patches were repaired, and the net gain is within noise, which our plan counts as failure.

4 Related work

Measuring self-preference. Panickssery et al. (2024) link a judge's preference for its own summaries to how well it recognizes them. Roytburg et al. (2026) compare a judge's own wrong answers with other models' wrong answers to the same question, and much of the measured self-preference goes away. Pombal et al. (2026) match the other writers on ability, and the preference shrinks but remains. Kozman et al. (2026) report that a model approves its own errors more often and that stronger generators' errors are harder to catch. We apply the same match to Gemma 4 and repository patches and find the same.

Verifying strong writers. Zhou et al. (2025) find that weak generators' errors are easier to detect than strong ones'. Khullar et al. (2026) find that one model rates a SWE-bench patch higher when it is framed as its own earlier answer, a cue our judges never see.

Tests as a check. In test-based program repair, test suites are often too weak, so repair tools produce many incorrect patches that pass them (Xiong et al., 2017). Our wrong fixes are this class. When code changes, some tests must change too, and Shang et al. (2026) find that coding agents identify those tests with an F1 below 50%. ImpossibleBench (Zhong et al., 2025) edits SWE-bench tests to conflict with the issue and tells the agent to put the issue first. Our conflicts come from the repositories' own tests, and our repair prompt asked for both without ranking them.

5 Limitations

The count across public models covers one agent. SWE-bench's list holds only tests from the files the reference pull request touches, so a wider test run could catch more wrong fixes, though our wider check stopped only 4 of Gemma's 44. In Gemma's held-out agent runs, the repository's Python environment was not active, so Gemma often could not import the code. This lowers its resolve rate and may produce more crash patches, which the repair fixes most easily. The repair's harm rests on 23 correct patches, and each Lite repair condition ran once.

6 Conclusion

A judge that seems to favor its own patches may partly be favoring strong writers. For Gemma 4 31B, about half of the measured preference remains against equally strong writers, and Gemma gives their failing patches the top score as often as its own. For an agent, the risk is Gemma's trust in wrong fixes, which neither its own checks nor the existing tests catch. Most failing patches of public agents pass the existing tests too, more so for stronger agents. An agent built on Gemma should not let Gemma's score decide that a patch is finished. When it adds the existing tests, it also needs a rule for which tests the issue may change, because on about one issue in ten the correct fix breaks one.

7 Data, code and compute

The GitHub repository holds the paper, supplement, code, plans and judge answers, and the Hugging Face dataset holds the agent runs and prompts. Running `./reproduce.sh main` regenerates every table in this paper from the released data, without a GPU. Two parts are meant for reuse: the 45 issues whose reference fix breaks an outdated test, and the equally strong writers' failing patches with nine judges' scores, which cost $5.29 to score. The project used about 113 GPU-hours, about $191 at RunPod on-demand prices.

References

Wei-Lin Chen, Zhepei Wei, Xinyu Zhu, Shi Feng and Yu Meng. 2025. *Do LLM Evaluators Prefer Themselves for a Reason?* https://arxiv.org/abs/2504.03846.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press and Karthik Narasimhan. 2023. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* https://arxiv.org/abs/2310.06770. ICLR 2024 (arXiv comment, https://openreview.net/forum?id=VTF8yNQM66).

Dipika Khullar, Jack Hopkins, Rowan Wang and Fabien Roger. 2026. *Self-Attribution Bias: When AI Monitors Go Easy on Themselves*. https://arxiv.org/abs/2603.04582.

Johnny Kozman, Arushi Waddepalli and Tilak Parajuli. 2026. *Neither Blinding Nor a Jury, Neither Capability Nor Strategy: What Brings a Verifier to Reliability*. https://neurips.cc/virtual/2026/173082. NeurIPS 2026 workshop "Who Verifies the Agents? Toward Reliable Agent Development" (poster).

MariusHobbhahn. *swe-bench-verified-mini*. https://huggingface.co/datasets/MariusHobbhahn/swe-bench-verified-mini. Hugging Face dataset; revision b316c349; accessed 2026-10-05.

Arjun Panickssery, Samuel R. Bowman and Shi Feng. 2024. *LLM Evaluators Recognize and Favor Their Own Generations*. https://arxiv.org/abs/2404.13076.

José Pombal, Ricardo Rei and André F. T. Martins. 2026. *Self-Preference Bias in Rubric-Based Evaluation of Large Language Models*. https://arxiv.org/abs/2604.06996.

Dani Roytburg, Matthew Bozoukov, Matthew Nguyen, Jou Barzdukas, Mackenzie Puig-Hall and Narmeen Oozeer. 2026. *Are LLM Evaluators Really Narcissists? Sanity Checking Self-Preference Evaluations*. https://arxiv.org/abs/2601.22548.

ICML 2026 Main.

Ye Shang, Quanjun Zhang, Haichuan Hu, Chunrong Fang, Liang Xiao and Zhenyu Chen. 2026. *Breaking, Stale, or Missing? Benchmarking Coding Agents on Project-Level Test Evolution*. https://arxiv.org/abs/2605.06125.

SWE-agent. *mini-SWE-agent*. https://github.com/SWE-agent/mini-swe-agent. GitHub repository; accessed 2026-10-05.

SWE-bench. *SWE-bench experiments*. https://github.com/SWE-bench/experiments. GitHub repository; patches from the swe-bench-submissions bucket; revision 40f164d5; accessed 2026-10-05.

SWE-bench. *SWE-bench Multilingual*. https://huggingface.co/datasets/SWE-bench/SWE-bench_Multilingual. Hugging Face dataset; revision 846e647b; accessed 2026-11-07.

Chunqiu Steven Xia, Yinlin Deng, Soren Dunn and Lingming Zhang. 2024. *Agentless: Demystifying LLM-based Software Engineering Agents*. https://arxiv.org/abs/2407.01489.

Yingfei Xiong, Xinyuan Liu, Muhan Zeng, Lu Zhang and Gang Huang. 2017. *Identifying Patch Correctness in Test-Based Program Repair*. https://arxiv.org/abs/1706.09120. ICSE 2018 (arXiv comment).

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan and Ofir Press. 2024. *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. https://arxiv.org/abs/2405.15793.

Ziqian Zhong, Aditi Raghunathan and Nicholas Carlini. 2025. *ImpossibleBench: Measuring LLMs' Propensity of Exploiting Test Cases*. https://arxiv.org/abs/2510.20270. ICLR 2026 (proceedings page https://iclr.cc/virtual/2026/poster/10009390, https://openreview.net/forum?id=SeO4vyAj7E).

Yefan Zhou, Austin Xu, Yilun Zhou, Janvijay Singh, Jiang Gui and Shafiq Joty. 2025. *Variation in Verification: Understanding Verification Dynamics in Large Language Models*. https://arxiv.org/abs/2509.17995. ICLR 2026 (arXiv comment).